Mon 6 Jan 2025

CSC 336 Numerical Methods.

- introduce
- This course provides an introduction to numerical / scientific computing.
- why chalk + talk : slows down; easier to adapt; more hands-on

Q: • Why should you take this course?

In many of your other courses, you have developed mathematical models to describe different processes.

These models are useful because it is much simpler to experiment with a mathematical model of say an aircraft wing than it is to build a new wing for each design, and then test that wing in a wind tunnel.

However, math models usually don't have closed form solutions - you often can't just write down the answer - so we need to find approximate solutions to the models.

Computer approximations are necessary since accurate solutions often require lots of data (i.e., memory) and lots of computations (i.e., cpu time)

You've all taken math courses. An obvious question to ask is:

Q: Can't we just take out our math textbooks, open our laptop and program up the math techniques that we've learned?

math text + laptop + code it up $\overset{?}{\Rightarrow}$ accurate answer to problem

Unfortunately its not always so simple:

Motivating example

In first year calculus you learned that the exponential function e^x has the Maclaurin series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{i=0}^{\infty} \frac{x^i}{i!}.$$

You can show that this series converges for all values of x. Beautiful!

Let's program this up and compute values of e^x!

issue: Can't compute to ∞.

proposal: Define the partial sum $S_n = \sum_{i=0}^{n} x^i/i!$ and stop computing when the partial sums have converged to 8 digits after decimal point. $(|S_n - S_{n-1}| < 1\times10^{-8})$

I've programmed this up. See Quercus for the code + observations.

Here are some observations for various values of x.

x	0	1	10	20	40
# terms in sum for "convergence"	2 ← min possible/required	13	42	69	104

Observations:

better alg:

find k s.t. $r = x/k$ exact. $|x/k| < 1$ compute $e^{x/k}$ then $e^x = (e^{x/k})^r$

e.g. divide out power of 2.

The last exponent in the last expression should be k not r.

<u>note:</u> The running time (as measured by the # terms required) depends very much on the value of x. This is not good. It would be better to derive a technique that takes the same time for any input.

(This is one thing that researchers in this area do. Try to derive new algorithms with nicer convergence properties.)

Also: For $x = -30$, the program converges after including 97 terms to $\approx -6.0 \times 10^{-5}$.

For $x = -40$, the program converges after including 124 terms to $\approx 5.9 \times 10^{0}$.

Comments? Both of these results are garbage because $0 < e^x < 1$ for $x < 0$!

Lesson from the example

Math textbook techniques don't always provide good computational algorithms.

One of the goals of the course is to show you <u>new</u> computational algorithms and to discuss why they are better.

$\longrightarrow$ Course topics + syllabus.

The topics that we will consider include:

① Computer Arithmetic and Computational Errors
(Chapter 1 of Heath)

In this section of the course we'll discuss properties of computer arithmetic and see that floating point computation of expressions like

$$(a+b)-a$$

often don't give b.

We'll see some of the pitfalls that result.

We'll discuss two important concepts:

- the conditioning of a mathematical problem
- the numerical stability of an algorithm.

This general background material will be used in the rest of the course.

② Solving Systems of Linear Equations
(Chapter 2 of Heath)

Here the problem is to solve the system of n linear equations in n unknowns given in matrix form as

$$\text{solve} \quad \underset{n\times n}{A}\ \underset{n\times 1}{\underline{x}} = \underset{n\times 1}{\underline{b}}$$

We'll see that the Gaussian Elimination process that you learned in linear algebra courses needs to be modified to become a robust and reliable algorithm

③ Solving Nonlinear Equations (Chapter 5 of Heath)

Here the problem is to find solutions x to a general equation of the form

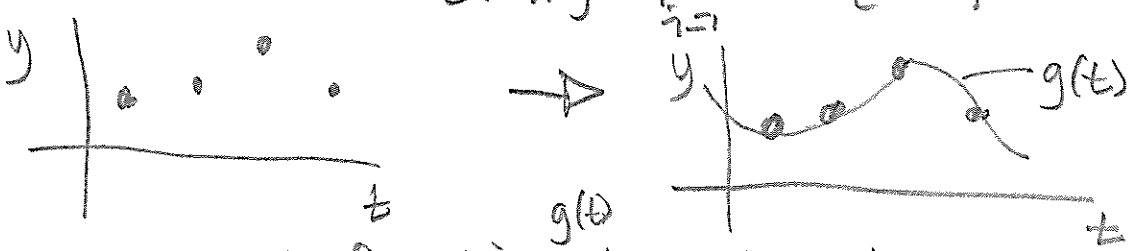
$$f(x) = 0$$
$$\text{or} \quad g(x) = c$$
$$\text{or} \quad f(x) = g(x)$$

We'll analyze various iterative methods (bisection method, secant method, Newton's method)

and will learn
- when to apply a particular method
- how to build reliable and efficient software for solving these problems.

④ Interpolation (Chapter 7 of Heath)

Here the problem is: "Given a set of data observations $\{(t_i, y_i)\}_{i=1}^{n}$ or $\{(t_i, f(t_i))\}_{i=1}^{n}$



derive a simple function $g(t)$ that fits them. We could use such a function to
- predict intermediate values.
- provide a compressed representation of the function

We'll see that some obvious algorithms are not always very accurate.

$\boxed{\text{Syllabus review}}$ → see Quercus

prerequisites:

CSC : so I know that you know how to implement an algorithm in a computer program

Calculus: so I know you know single variable calculus
- may want to review differentiation and integration rules

algebra : elementary linear algebra
- need to be comfortable with matrices, vectors and norms.

Assignments
- some "pencil + paper" exercises and some programming exercises.

Please read the details!

Sept 1983
- goal : get scientists to stop writing all code from scratch and use state of the art software

need to learn
- how to interpret the results.
- how to choose algorithm

Jan 2025
- goal : understand what it is that you've downloaded

still need to learn
- how to choose algorithm to use
- how to interpret the results.

Before I stop, let me tell you a story.

In the spring of 2019, I was in Ottawa and use the app for their transportation system. I got this message

- note the size of the update

approx 3,889,999.9913 ...

- issues • not very approximate
  - what's .9913 of a byte?
  - do I need to know size to that level of detail?

I hope that after you complete the course you will think more about numbers and format them in a sensible way!

ROGERS LTE 6:10 PM 88%

OC Transpo

Schedule Update Available

An updated schedule is available for download and is approximately 3.889999913051724398 MB in size

?

parts of a byte

Update Now

Later